Contents

1 Dexter CLI System Architecture

1.1 Table of Contents

8. Tool System
9. Storage & Persistence
10. Event System

1.2 High-Level Architecture

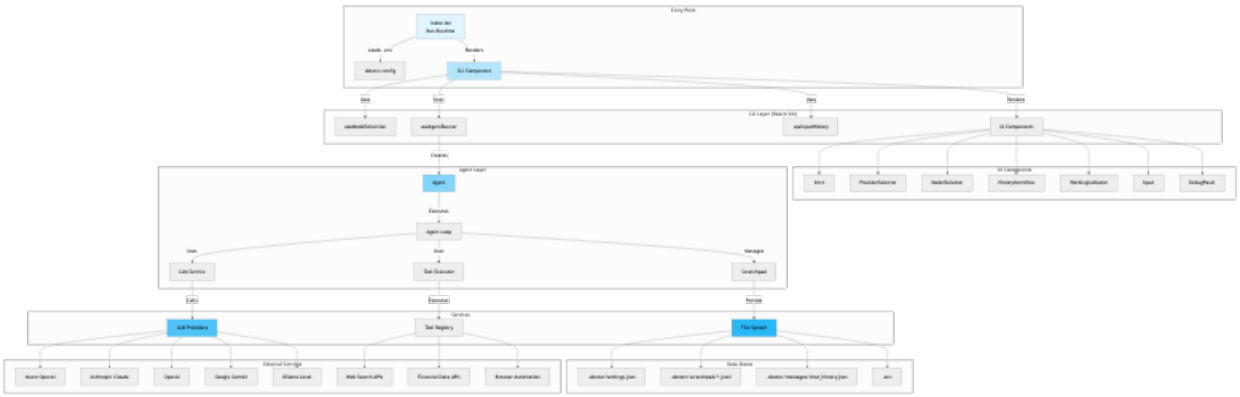

Figure 1: Diagram 1

1.3 Entry Point & CLI Flow

1.4 Component Hierarchy

1.5 Agent Execution Flow

1.6 State Management

1.7 Data Flow: User Input → Agent → Output

1.8 LLM Integration

1.9 Tool System

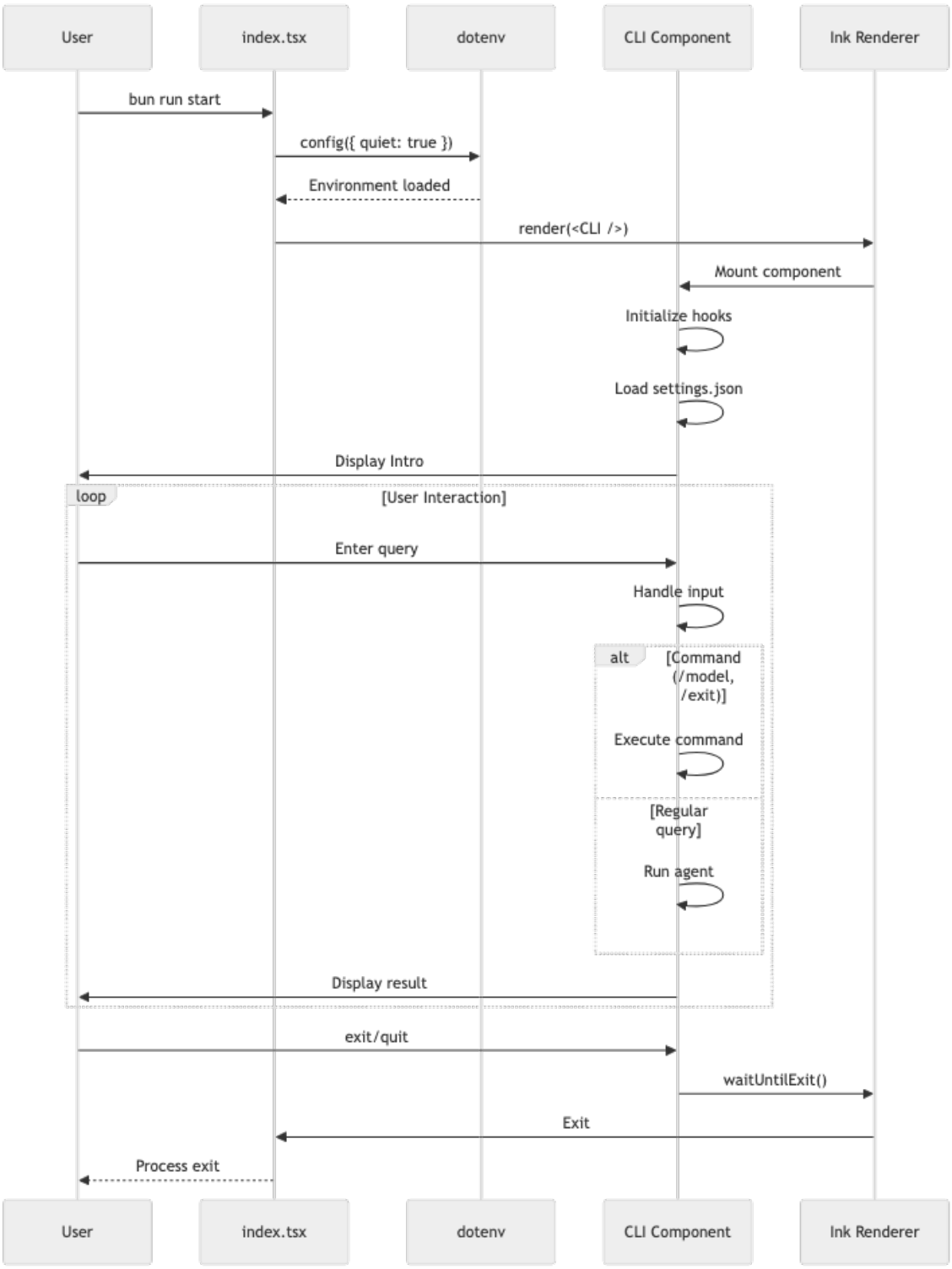

Figure 2: Diagram 2

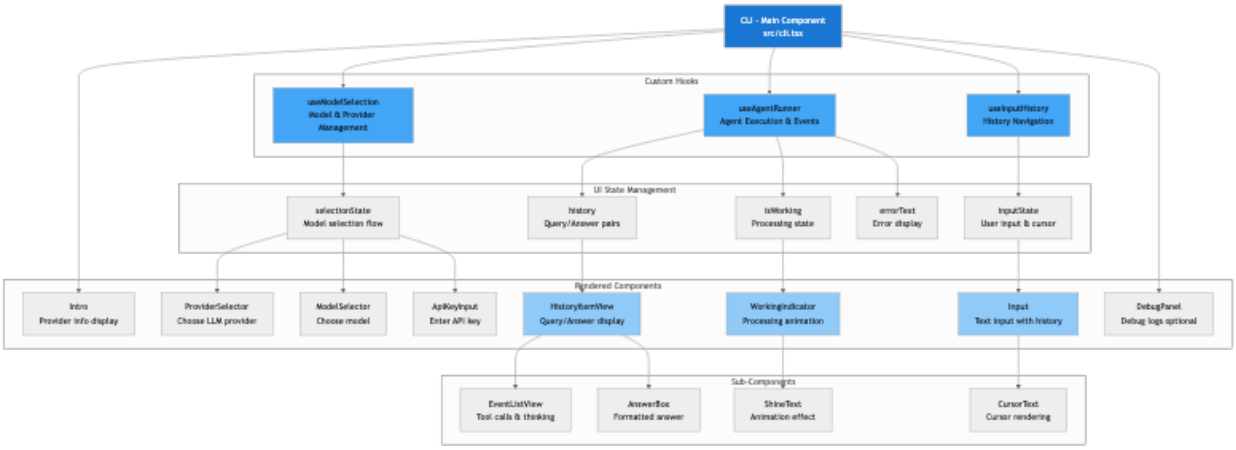

Figure 3: Diagram 3

1.10 Storage & Persistence

1.11 Event System

1.11.1 Event Types

```
type AgentEvent =
  | ThinkingEvent          // LLM reasoning text
  | ToolStartEvent         // Tool execution initiated
  | ToolProgressEvent      // Real-time progress from tool
  | ToolEndEvent           // Tool completed successfully
  | ToolErrorEvent         // Tool failed with error
  | ToolLimitEvent         // Tool call limit warning
  | ContextClearedEvent    // Old context removed
  | AnswerStartEvent       // Final answer generation started
  | DoneEvent              // Complete with answer & metrics
```

1.12 Keyboard Shortcuts & Commands

Input	Action
`exit`, `quit`	Exit application
`/model`	Start model selection flow
`Esc`	Cancel current action/execution
`Ctrl+C`	Cancel or exit
`↑` / `↓`	Navigate input history
`Shift+Enter`	Multi-line input
`Ctrl+A`	Move to start of line
`Ctrl+E`	Move to end of line
`Alt+←` / `Alt+→`	Word navigation

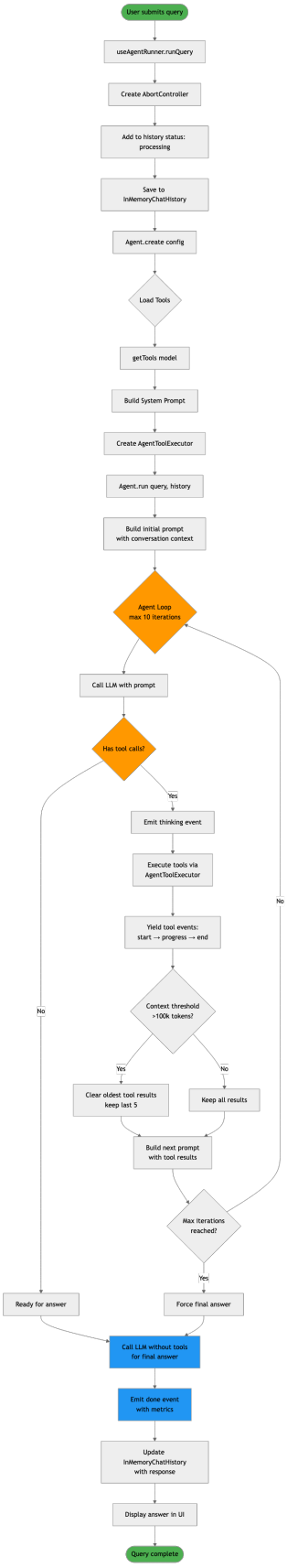

Figure 4: Diagram 4

Diagram 5

Figure 5: Diagram 5

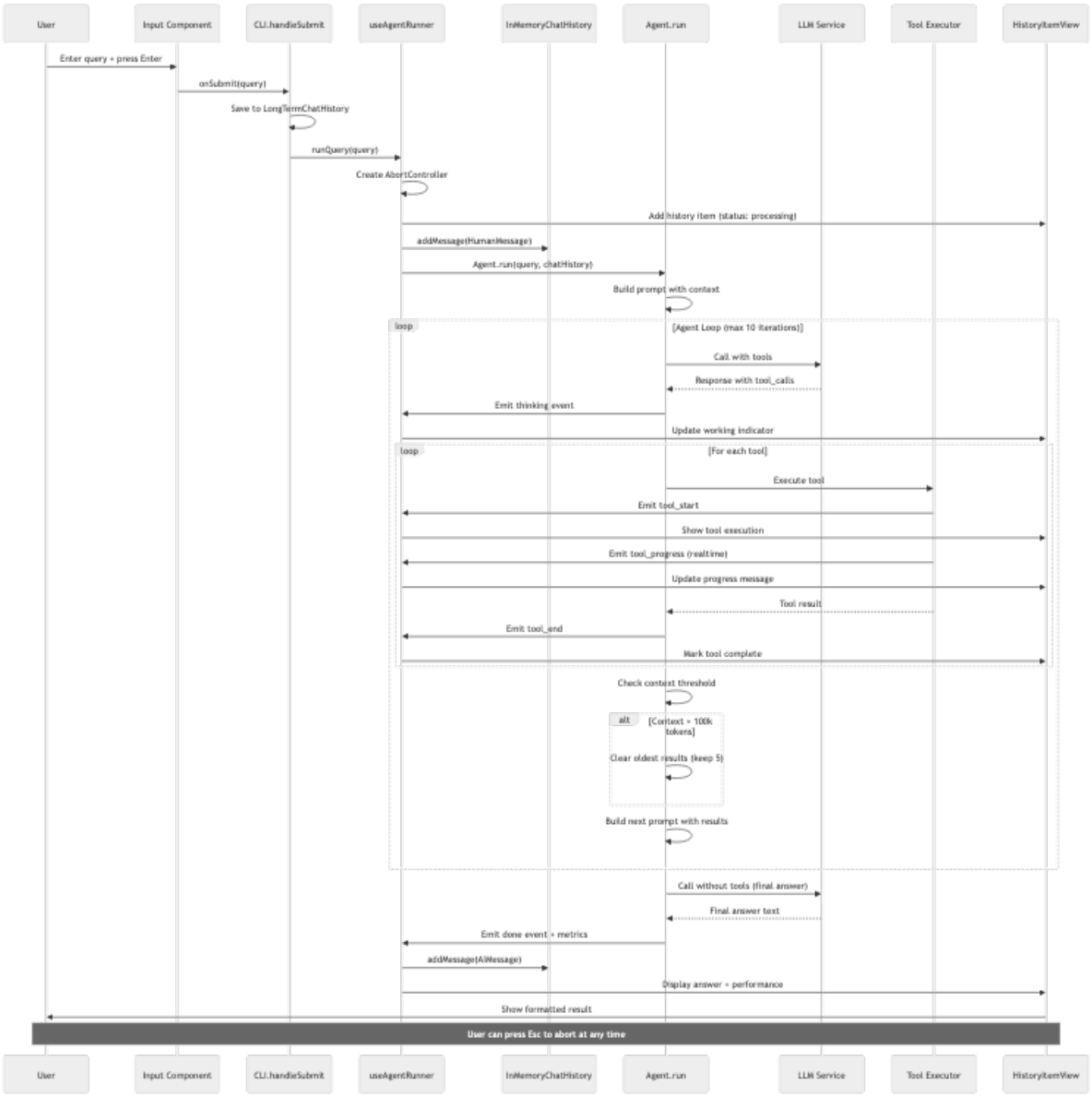

Figure 6: Diagram 6

Figure 7: Diagram 7

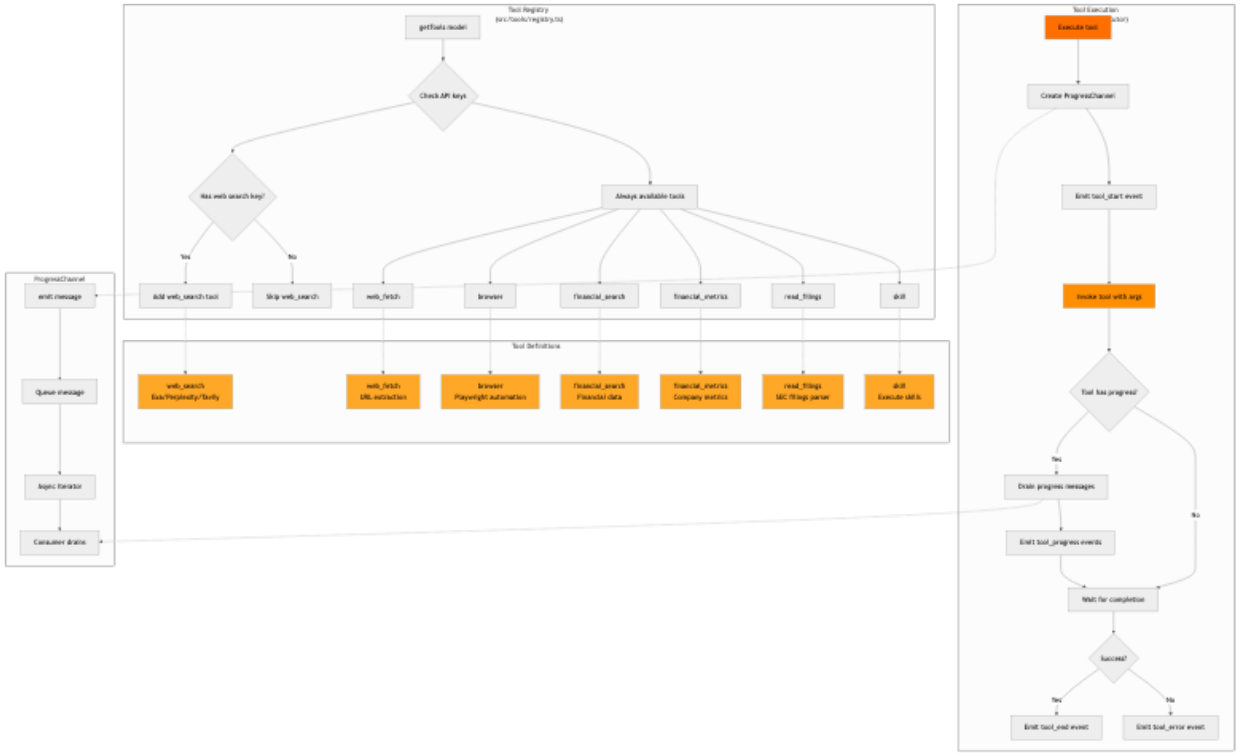

Figure 8: Diagram 8

Figure 9: Diagram 9

Figure 10: Diagram 10

1.13 Performance Optimizations

1.13.1 1. Anthropic Prompt Caching

- System prompt marked with `cache_control: { type: 'ephemeral' }`
- 90% cost reduction on subsequent calls
- Implemented in `buildAnthropicMessages()`

1.13.2 2. Azure Token Caching

- Tokens cached for 5 minutes
- Reduces auth overhead
- Automatic refresh before expiry

1.13.3 3. Context Management

- Threshold: 100k tokens
- Keeps last 5 tool results
- Clears oldest results when exceeded
- Full context for final answer

1.13.4 4. Tool Result Summarization

- LLM summarizes large tool outputs
- Reduces context size
- Preserves key information

1.13.5 5. InMemoryChatHistory Relevance

- Caches relevance scores
- Only selects relevant past messages
- Reduces prompt size for multi-turn

1.14 Error Handling & Safety

1.14.1 1. Graceful Cancellation

```
AbortController → agent.run(signal)
                → tool.invoke(signal)
                → Mark as 'interrupted'
```

1.14.2 2. Retry Logic

- Exponential backoff
- Max 3 attempts per LLM call
- Logs each failure attempt

1.14.3 3. Soft Limits

- Tool call warnings (not blocks)
- Query similarity detection
- Informs LLM via prompt

1.14.4 4. Error Display

- Tool errors shown in event list
- Agent continues execution
- Final error state if critical

1.15 Architecture Highlights

1.15.1 Strengths

1. **Real-time Event Streaming** - Async generator pattern for instant UI updates
2. **Modular Hook Design** - Separation of concerns (model, agent, history)
3. **Multi-provider Support** - Easy to add new LLM providers
4. **Graceful Degradation** - Continues on tool errors, soft limits
5. **Context Management** - Intelligent clearing for long conversations
6. **Comprehensive Persistence** - Settings, history, scratchpad all saved
7. **Developer Experience** - Ink/React for terminal UI, TypeScript
8. **Cancellation Support** - User can abort at any time
9. **Progress Feedback** - Real-time tool progress via ProgressChannel
10. **Flexible Tool System** - Easy to add new tools

1.15.2 Key Patterns

- **React Hooks** for state management
- **Async Generators** for event streaming
- **AbortController** for cancellation
- **Ink Components** for terminal rendering
- **JSONL Logs** for debugging
- **TokenManager** for credential management
- **ProgressChannel** for real-time updates

1.16 File Structure Summary

```
src/
  index.tsx                  # Entry point (Bun runtime)
  cli.tsx                    # Main CLI component (React/Ink)

  agent/
     agent.ts                # Agent class with execution loop
     prompts.ts              # System prompts
     scratchpad.ts           # JSONL logging & context management
```

```
    token-counter.ts         # Token usage tracking
    types.ts                 # Agent event types

  model/
    llm.ts                   # LLM integration (getChatModel, callLlm)
    token-manager.ts         # Keychain + env credential management
    azure-openai-models.ts   # Azure constants

  tools/
    registry.ts              # Tool registration (getTools)
    web-search.ts            # Search tools (Exa/Perplexity/Tavily)
    web-fetch.ts             # URL fetching
    browser.ts               # Playwright automation
    financial-search.ts      # Financial data
    skill.ts                 # Skill execution

  ui/
    model-selector.tsx       # Provider/model selection
    history-item-view.tsx    # Query/answer display
    answer-box.tsx           # Formatted answer
    agent-event-view.tsx     # Tool/thinking events
    working-indicator.tsx    # Processing animation
    input.tsx                # Text input with history

  hooks/
    use-model-selection.ts   # Model selection state
    use-agent-runner.ts      # Agent execution state
    use-input-history.ts     # History navigation

  utils/
    in-memory-chat-history.ts      # Conversation context
    long-term-chat-history.ts      # Persistent history
    progress-channel.ts            # Real-time progress
    settings.ts                    # Settings persistence
    logger.ts                      # Logging

  providers.ts                # Provider registry

.dexter/
  settings.json               # Provider & model config
  scratchpad/
    timestamp_hash.jsonl      # Per-query logs
  messages/
     chat_history.json        # All conversations
```

1.17 Technology Stack

Layer	Technology
Runtime	Bun
Language	TypeScript
UI Framework	React + Ink (terminal rendering)
LLM Integration	LangChain
State Management	React Hooks + Refs
Storage	JSON/JSONL files
Authentication	Azure Identity, TokenManager
Web Automation	Playwright
Async Patterns	Async generators, EventEmitter

1.18 Extending the Architecture

1.18.1 Adding a New Tool

1. Create tool in `src/tools/`
2. Register in `src/tools/registry.ts`
3. Optionally implement ProgressChannel for real-time updates

1.18.2 Adding a New LLM Provider

1. Add factory to `MODEL_FACTORIES` in `src/model/llm.ts`
2. Register provider in `src/providers.ts`
3. Update UI options in `PROVIDERS` array

1.18.3 Adding New UI Component

1. Create component in `src/ui/`
2. Integrate in `src/cli.tsx`
3. Use Ink's Box/Text components for layout

Last Updated: 2026-02-14 **Architecture Version:** 2026.2.14